

# C언어 기말 프로젝트 보고서

## 행맨(HangMan)의 구현

19105332 조동영

대구가톨릭대학교 컴퓨터소프트웨어학부

---

### 1. 서론

본 프로젝트는 C언어의 기본 문법과 표준 라이브러리, 그리고 Windows API를 활용하여 텍스트 기반 콘솔 게임인 '행맨(Hangman)'을 구현하는 것을 목표로 합니다.

본 프로그램은 과제에서 요구하는 기본 행맨 게임(알파벳 맞추기)의 규칙에서 더 나아가, \*\*'영어 명언 문장 빈칸 단어 맞추기'\*\*라는 독창적인 아이디어를 적용하여 게임성을 혁신적으로 변형하였습니다.

이는 사용자가 단순한 암기나 운에 의존하는 것이 아니라, 문장의 맥락과 단어 지식을 활용하여 문제를 해결하도록 유도합니다.

특히, 기존의 행맨 게임이 가진 '교수형'이라는 으스스한 컨셉을 극대화하기 위해 할로윈 테마에 맞춘 3가지(호박, 해골, 허수아비)의 아스키 아트(ASCII Art) 캐릭터를 디자인하고, PlaySound 함수를 활용하여 \*\*으스스한 배경음악(BGM)\*\*을 삽입하였습니다.

이러한 멀티미디어적 요소와 독창적인 규칙 변형은 본 프로젝트의 가장 큰 특징이자 강점입니다.

사용자는 총 10개의 퀴즈 문장(영어 속담 및 명언)을 순서대로 제공받으며, 문장 속 '\*\*\*'로 표시된 빈칸에 해당하는 단어를 유추하여 입력하게 됩니다.

주요 기능으로는 캐릭터 선택 기능, BGM 재생, Beep 함수를 이용한 정답/오답 효과음 출력, 그리고 `_getch` 함수를 활용한 '**다시하기**' 기능 등이 구현되었습니다.

또한, `fgets`를 사용하여 엔터만 입력하는 등의 비정상적인 입력 상황을 방지하고 프로그램의 안정성을 확보하였습니다.

본 보고서는 프로그램의 개발 순서에 따라 다음 내용을 기술할 것입니다.

**2장**에서는 C언어 표준 라이브러리 및 BGM/효과음 재생에 사용된 Windows API(`windows.h`, `conio.h`) 등 **관련 연구**를 기술할 것입니다.

**3장**에서는 게임의 핵심 로직(생존/패배 판정)과 퀴즈 데이터 및 아스키 아트 캐릭터의 자료 구조 등 **프로그램 설계**에 대해 설명드릴 것입니다.

**4장**에서는 BGM과 효과음의 동시 출력 방법, `fgets`와 `_getch`를 활용한 입력 최적화 등 **프로그램 구현**의 핵심 기술을 다룰 것입니다.

**5장**에서는 실제 **프로그램의 동작과 결과**를 캡처 화면과 함께 제시하고

**6장**에서는 구현 중 발생한 버그(LNK 오류, 입력 버퍼 문제)와 해결 과정을 **분석 및 토의**할 것입니다.

마지막으로 **7장**에서는 프로젝트 전체를 **결론**짓겠습니다.

---

## 2. 관련 연구

이번 프로젝트는 C 언어의 기본 문법은 물론, 콘솔 기반 프로그램의 한계를 보완하기 위한 Windows API의 활용, 문자열 처리, 멀티미디어 출력 등 다양한 기술 요소에 대한 이해를 필요로 하였습니다.

특히 게임 도중 배경음악을 재생하거나 정답·오답 효과음을 출력하기 위해 Windows에서 제공하는 PlaySound 및 Beep 기능을 사용하였으며, 이는 일반적인 C 언어 수업에서는 다루지 않는 영역으로 별도의 학습이 요구되었습니다.

또한 프로그램 흐름을 보다 자연스럽게 만들기 위해 `_getch()`를 사용하여 엔터키 없이도 입력을 즉시 처리하도록 구현했으며, 이는 사용자 경험 (UX)을 개선하는 데 중요한 역할을 했습니다.

행맨 게임의 특성상 퀴즈 문장에서 특정 단어를 추출·비교하는 과정이 필요하기 때문에 문자열 처리 또한 중요한 요소였습니다.

이를 위해 `strcmp`, `strlen`, `fgets` 등의 함수를 사용하여 비정상 입력을 방지하고 안정적인 입력 제어 로직을 구현하였습니다.

콘솔 화면을 지우거나 출력 속도를 조절하기 위한 `system("cls")`, `Sleep()`과 같은 Windows 시스템 함수들도 함께 사용되었으며, 이를 통해 단순 텍스트 출력 이상의 연출 효과를 구현할 수 있었습니다.

이러한 기능들을 유기적으로 결합함으로써 단순한 텍스트 기반에 머무르지 않고, 사운드·애니메이션 연출·입력 제어가 포함된 하나의 "게임"으로 완성도 있게 구현하는 것이 본 프로젝트의 핵심이었습니다.

본 프로젝트는 C 언어의 기본기를 넘어 Windows 환경에서의 API 활용 능력을 평가할 수 있는 기회가 되었으며, 콘솔 프로그래밍의 확장 가능성을 직접 확인할 수 있는 경험이 되었습니다.

---

### 3. 프로그램 설계

프로그램은 크게 **인트로 화면**, **게임 진행 화면**, **최종 결과 화면**의 흐름으로 구성되어 있습니다.

처음 실행하면 인트로 화면을 통해 게임 규칙과 캐릭터 선택 방법을 안내하고, 사용자가 캐릭터를 선택하면 본격적인 퀴즈 진행 화면으로 넘어갑니다.

퀴즈는 총 **10개 문제**로 구성되어 있으며, 각 문제는 빈칸이 포함된 영어 문장과 힌트가 함께 제시됩니다.

사용자는 직접 단어를 입력해 정답을 맞추게 되고, 정답 여부에 따라 점수가 올라가거나 틀린 횟수가 증가합니다.

문제를 틀리면 선택한 캐릭터의 행맨 그림이 단계별로 완성되며, 정답·오답 상황에 맞춰 효과음이 재생되어 게임 몰입감을 높였습니다.

10개의 문제를 모두 풀면 최종 화면으로 넘어가게 되며, 맞춘 문제 수와 틀린 횟수를 기준으로 게임의 **승리 / 패배**가 결정됩니다.

결과 화면에서는 간단한 평가와 함께 “다시 하기” 선택지가 제공되어 사용자가 원하는 경우 즉시 새 게임을 시작할 수 있습니다.

전체 화면 전환은 콘솔 기반이기 때문에 각 기능을 독립된 함수로 나누어 자연스럽게 흐름이 이어지도록 설계했습니다.

입력, 캐릭터 출력, 문제 진행, 결과 출력 같은 요소를 분리해  
코드 가독성과 유지보수성을 높이려는 점도 고려했습니다.

---

## 4. 프로그램 구현

이번 프로젝트는 Microsoft Visual Studio 2022 환경에서 C언어를 사용하여 구현하였으며  
기본 행맨 게임의 틀을 바탕으로 다양한 기능을 추가하여 완성도를 높였습니다.

특히 캐릭터 아스키 아트와 배경음악, 효과음 처리 등 콘솔 환경에서는 구현하기 어려운 부분도 직접 다뤄 게임의 몰입도를 높였습니다.

프로그램의 주요 기능은 하나의 파일(HangmanGame.cpp)에 구현하였으며  
각 기능을 별도의 함수로 나누어 관리하고 유지보수성을 높였습니다.

호박(Pumpkin), 해골(Skeleton), 허수아비(Scarecrow) 3종 캐릭터를  
실패 단계 6단계에 맞추어 직접 디자인하였습니다.

캐릭터는 3차원 문자열 배열로 구성되어 있으며  
콘솔 출력 시 글자 폭이 어긋나지 않도록 정렬에 신경을 썼습니다.

" 등 일부 특수 문자가 콘솔에서 깨지는 문제는 \*, ~, H, O 등 안전한 문자로 대체하여 해결하였습니다.

캐릭터 선택 기능에서는 select\_character() 함수에서는 fgets()를 사용하여 입력을 받으며,  
1, 2, 3 이외의 값이 입력될 경우 다시 선택하도록 구현하였습니다.

이를 통해 비정상적인 입력으로 프로그램 흐름이 꼬이는 문제를 방지하였습니다.

**퀴즈 진행 로직에서** play\_game() 함수는 10개의 문제를 순서대로 제공하며, 사용자가 틀리거나 공백을 입력하면 즉시 정답을 공개하고 다음 문제로 넘어가도록 구성하였습니다.

문제는 문장, 정답, 힌트가 세트로 구성되어 있어, 단순한 게임 진행뿐만 아니라 학습 효과도 함께 제공합니다.

멀티미디어 구현 및 입력 처리 최적화 부분에서 콘솔 환경에서 배경음악과 효과음을 동시에 구현하는 것은 어려움이 있으나 프로젝트 몰입도를 높이기 위해 다양한 방법을 적용하였습니다.

BGM은 Pixabay에서 할로윈 테마 음원을 확보한 후 Audacity를 이용하여 템포, 볼륨, 음높이 등을 편집하여 게임 분위기에 맞게 조정하였습니다.

PlaySound() 함수를 사용하여 게임 시작 시 음악이 비동기로 반복 재생되도록 구현하였습니다.

또한 L 접두사를 사용하여 문자열 타입 불일치 오류(C2664)를 방지하였습니다.

BGM 재생 중에도 효과음이 출력되도록 Beep() 함수를 사용하여 정답과 오답음 을 분리했습니다.

정답 시에는 '딩동댕' 멜로디(Beep(523, 120); Beep(659, 120); Beep(784, 150);),  
오답 시에는 낮은 톤의 경고음(Beep(261, 300); Beep(261, 300);)을 출력하도록 구현하였습니다.

이를 통해 BGM과 효과음이 동시에 출력되면서도 높은 완성도를 유지할 수 있었습니다.

입력 최적화 부분은 사용자 입력은 scanf() 대신 fgets()를 사용하여 공백과 엔터까지 안정적으로 읽도록 하였습니다.

입력 문자열 끝의 \n을 제거하여 strcmp() 비교가 정확하게 이루어지도록 처리하였습니다.

게임 종료 후 '다시 하기' 기능은 \_getch() 함수를 사용하여 숫자 키를 누르는 즉시 화면이 전환되도록 구현하여, 콘솔 환경에서도 사용자 편의성을 최대한 고려하였습니다.

#### [별첨] BGM 재생 코드 예시

```
#pragma comment(lib, "winmm.lib") // 윈도우 멀티미디어 라이브러리 링크

int main() {
    // BGM 재생 (무한 반복, 비동기)
    PlaySound(L"bgm.wav", NULL, SND_ASYNC | SND_LOOP | SND_FILENAME);

    int play_again = 1;
    while (play_again == 1) {
        // 게임 루프 진행...
    }

    // 게임 종료 시 BGM 정지
    PlaySound(NULL, 0, 0);
    return 0;
}
```

---

## 5. 프로그램의 동작과 결과

프로그램을 실행하면 가장 먼저 BGM과 함께 메인 화면이 나타나며, 플레이어는 3가지 캐릭터 중 하나를 선택하게 됩니다.

캐릭터를 선택하면, 선택한 캐릭터와 함께 총 10문제가 차례대로 출제됩니다.

플레이어는 각 문제에서 빈칸(\*)에 들어갈 단어를 입력해야 하며, 오답 시 비프음과 함께 캐릭터 그림이 한 단계씩 완성됩니다.

오답이 6회 누적되면 캐릭터가 완성되어 게임에서 패배하게 됩니다.

총 10문제 중 절반 이상을 맞추면 승리로 판정되며, 절반 미만을 맞출 경우에는 1. 다시하기 / 2. 종료하기 화면으로 이동하여 플레이어가 선택할 수 있습니다.

프로그램을 실행하면 main() 함수에 의해 **bgm.wav** 파일이 비동기 무한 반복(SND\_LOOP)으로 재생되기 시작합니다. 초기 화면에서는 show\_intro() 함수가 호출되어 게임의 규칙 및 승패 조건을 사용자에게 설명합니다.

```
=====
[ 게임 설명 ]
=====
이 게임은 컴퓨터가 내는 10개의 문장 빈칸을 맞추는 게임입니다.

[패배 조건 1]
단어를 6번 틀려 캐릭터가 모두 그려지면 **패배**합니다.

[패배 조건 2 / 생존 조건]
6번 틀리기 전에 10문제를 모두 풀면,
맞춘 개수가 절반(6개) 이상일 경우 **생존**합니다.
절반을 맞추지 못하면(5개 이하) **패배**합니다.
=====

[ 계속하려면 아무 키나 누르세요... ]_
```

사용자가 아무 키나 입력하면 `getchar()` 함수가 반응하여 화면이 지워지고, 캐릭터 선택 단계로 넘어갑니다.

`select_character()` 함수가 호출되어 3가지 테마 캐릭터(호박, 해골, 허수아비)의 최종 형태를 미리보기로 출력합니다.

```
=====
행맨(HANGMAN) 게임 캐릭터 선택
=====

----- [1] 호박 (Pumpkin) -----
+---+
|   |
(oVo)
 /|\*
 / \
-----

=====

----- [2] 해골 (Skeleton) -----
+---+
|   |
(o)
 ~H~
  | |
-----

=====

----- [3] 허수아비 (Scarecrow) -----
      /  /  |
     /  /  | @
    /  /  |  \
   /  /  |   \
  /  /  |    \
 /  /  |     \
/  /  |      \#
-----

=====

[1] 호박 (Pumpkin)
[2] 해골 (Skeleton)
[3] 허수아비 (Scarecrow)

캐릭터를 선택하세요 (1, 2, 3): _
```

사용자는 '1', '2', '3' 중 하나를 선택합니다. 만약 **엔터만 입력하거나** 유효하지 않은 문자열을 입력할 경우, fgets()를 통한 입력 유효성 검사 루프에 의해 **다시 입력**하도록 강제로 화면 전환이 됩니다.

사용자가 캐릭터를 선택하면 화면이 전환 됩니다.

```
=====
행맨 (HANGMAN) 게임 캐릭터 선택
=====

----- [1] 호박 (Pumpkin) -----
+---+
|   |
| (oVo)
|  /|*
|  /  \
+---+
=====

----- [2] 해골 (Skeleton) -----
+---+
|   |
| (o)
|  ~H~
|  |  |
+---+
=====

----- [3] 허수아비 (Scarecrow) -----
+---+
| /  \ |
| /  \ |
| /  \ |
| /  \ |
| /  \ |
+---+
=====

[1] 호박 (Pumpkin)
[2] 해골 (Skeleton)
[3] 허수아비 (Scarecrow)

캐릭터를 선택하세요 (1, 2, 3):
잘못된 입력입니다. 1, 2, 3 중 하나를 정확히 입력하세요.
(잠시 후 다시 선택합니다....)
```

```

(oVo)
 /|*
 /  \
=====

----- [2] 해골 (Skeleton) -----
+---+
|   |
| (o)
|  ~H~
|  |  |
+---+
=====

----- [3] 허수아비 (Scarecrow) -----
+---+
| /  \ |
| /  \ |
| /  \ |
| /  \ |
| /  \ |
+---+
=====

[1] 호박 (Pumpkin)
[2] 해골 (Skeleton)
[3] 허수아비 (Scarecrow)

캐릭터를 선택하세요 (1, 2, 3): 3
==> 3번 캐릭터가 선택되었습니다!

[ 잠시 후 게임을 시작합니다... ]
```

선택된 캐릭터와 함께 `play_game()` 함수가 실행되며, 10개의 퀴즈 문장이 순서대로 출제됩니다. 매 문제마다 현재 맞춘 개수와 틀린 횟수가 명확히 표시됩니다.

## 정답 입력 시 (생존):

사용자가 정확한 정답 단어를 입력하면 strcmp를 통해 일치 여부를 확인하고, **\*\*'딩동댕' 멜로디 (Beep)\*\***와 함께 다음 문제로 넘어갑니다.

--- [ 문제 1 / 10 ] (맞춘 개수: 0 / 틀린 횟수: 0 / 6) ---

문제: With great power comes \*\*\* responsibility.

힌트: 스파이더맨 명대사



빈칸(\*)에 들어갈 단어를 입력하세요: great

[0] 정답입니다! 'great' 맞습니다!

[ 잠시 후 다음 문제로 넘어갑니다... (1초) ]

오답 입력 시 (캐릭터 진화):

사용자가 틀린 단어나 공백(엔터)을 입력하면, '땡!' 메시지와 함께 Beep 경고음이 출력됩니다.

이와 동시에 **\*\*정답은 '000' 입니다.\*\***라는 문구가 출력되어 정답을 확인하게 합니다.

wrong\_guesses가 1 증가하며 행맨 캐릭터의 아스키 아트가 한 단계 더 그려지고, 1초 후 다음 문제로 넘어갑니다.

— [ 문제 1 / 10 ] (맞춘 개수: 0 / 틀린 횟수: 0 / 6) —

문제: With great power comes \*\*\* responsibility.

힌트: 스파이더맨 명대사



빈칸(\*)에 들어갈 단어를 입력하세요: d

[X] 땡! 'd'가 아닙니다...

[!] 정답은 'great' 입니다.

[ 정답 확인 후 다음 문제로 넘어갑니다... (1초) ]

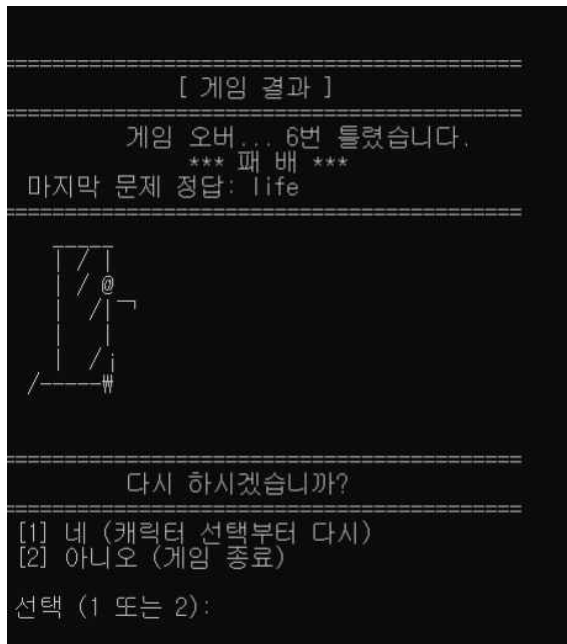
### 최종 결과 판정 및 재시작 기능

10문제를 모두 풀거나, 6번 틀려 캐릭터가 완전히 그려지면 게임 루프가 종료되고 최종 결과 화면이 출력됩니다.

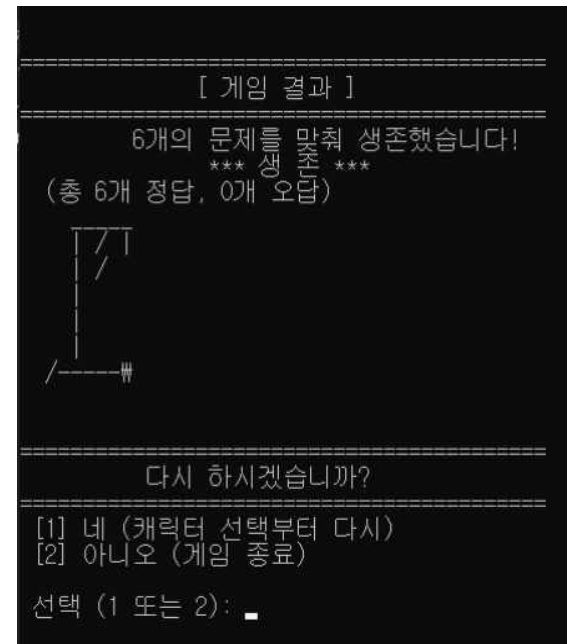
최종 결과 출력 후, main() 함수로 복귀하여 `_getch()` 함수를 통해 **재시작 여부**를 묻습니다.

'1'을 누르면 즉시 `show_intro()`로 복귀하여 게임이 재시작되고, '2'를 누르면 `PlaySound(NULL, 0, 0)`을 통해 BGM이 정지되고 프로그램이 종료됩니다.

(6번 틀리거나, 5개 이하 맞춘 경우)



(6개 이상 맞춘 경우)



---

## 6. 결과 분석 및 토의

본 장에서는 ‘행맨’ 프로그램의 실제 동작을 검토하고, 구현 과정에서 발생한 문제와 해결 과정을 분석하여 프로그램의 완성도를 평가합니다.

프로그램은 10개의 영어 명언 빈칸 맞추기 퀴즈가 설계대로 순차적으로 출제되며, strcmp를 활용한 정답 판정도 정확하게 작동합니다. 맞춘 문제가 6개 이상이면 게임 루프가 즉시 종료되어 조기 승리 판정이 가능하며, 이를 통해 게임의 속도감과 사용자의 몰입도를 높였습니다.

또한 BGM과 정·오답 효과음을 분리하여 출력함으로써, 콘솔 환경에서도 BGM이 끊기지 않고 효과음이 동시에 재생되도록 구현하였습니다. BGM은 Audacity를 사용해 템포와 음량을 조절하여 게임 테마와 조화를 이루도록 편집했습니다.

구현 과정에서 발생한 주요 문제와 해결 방법은 다음과 같습니다.

첫째, 입력 안정성 문제입니다. select\_character() 함수에서 scanf()를 사용하면 엔터 입력이 무시되어 무한 루프가 발생했으나, fgets()로 입력을 받고 문자열 끝의 \n을 제거하여 엔터 입력도 유효한 오답으로 처리함으로써 문제를 해결했습니다.

둘째, 비정상 입력 처리 문제입니다. 캐릭터 선택 시 숫자가 아닌 다른 문자가 입력될 경우, scanf()는 이를 읽지 못하고 입력 버퍼가 꼬였습니다. 이를 fgets()와 strcmp()로 "1\n", "2\n", "3\n"만 허용하도록 바꾸어 잘못된 입력은 재요청하도록 처리했습니다.

셋째, 재시작과 종료의 편의성 문제입니다. 게임 종료 후 ‘다시하기’ 선택 시 엔터를 눌러야 진행되는 불편함이 있었으나, \_getch()를 사용하여 키

입력 즉시 반응하도록 수정했습니다.

넷째, 링커 오류와 문자열 타입 불일치 문제입니다. PlaySound 사용 시 winmm.lib가 링크되지 않거나 문자열 타입이 맞지 않는 오류가 발생했으나, `#pragma comment(lib, "winmm.lib")`를 추가하고 파일명 앞에 L 접두사(L"bgm.wav")를 붙여 해결했습니다.

다만 C 언어 콘솔 환경에서는 외부 음원을 동시에 재생할 수 없는 한계가 있어, 정답/오답 효과음과 배경 BGM을 각각 다르게 설정하는 것은 불가능했습니다. 이로 인해 배경 BGM과 효과음 간 충돌을 완전히 피할 수 없다는 점은 다소 아쉬운 부분으로 남았습니다.

결과적으로, 본 프로그램은 기획 단계에서 설계한 10문제 퀴즈, 3종 캐릭터 선택, BGM과 효과음 재생, 재시작 루프 등 모든 기능을 안정적으로 구현했습니다.

특히 C 언어 콘솔 환경의 한계를 Windows API를 활용하여 해결함으로써, C 프로그래밍에 대한 이해와 실전 문제 해결 능력을 보여주는 프로젝트가 되었습니다.

---

## 7. 결론

이번 프로젝트를 통해 C언어의 기본 개념과 Windows API 활용 능력을 결합하여, 독창적인 규칙과 테마를 가진 콘솔 기반 '행맨' 게임을 성공

적으로 구현하였습니다.

“영어 명언 빈칸 맞추기”라는 차별화된 아이디어와 멀티미디어(BGM/SFX) 기능 구현을 통해 C 프로그래밍에 대한 깊이 있는 이해와 문제 해결 능력을 입증할 수 있었습니다.

C언어 환경의 한계를 극복하여 PlaySound, Beep, \_getch 등 Windows API를 활용하여, 콘솔 환경에서 기존에는 불가능했던 BGM과 효과음의 동시 출력 및 엔터 입력 없이도 반응하는 사용자 입력 최적화를 구현하였습니다.

프로그램 안정성 확보를 위해 fgets를 사용하여 scanf에서 발생할 수 있는 입력 버퍼 꼬임 문제를 방지하였고, 링커 오류(LNK1168, C2664)에 대해 분석과 대응을 통해 프로그램의 견고함을 높였습니다.

창의적으로 문제 해결을 하며 ‘6개 이상 정답 시 조기 생존’이라는 독창적인 규칙을 play\_game 루프 조건에 통합함으로써, 게임의 완성도와 몰입도를 한층 강화하였습니다.

저의 프로그래밍 소견과 미래 개발자로서의 역할에 대한 생각은 다음과 같습니다.

본 프로젝트를 수행하며, “더 이상 사람이 코딩만으로 프로그램을 만드는 시대는 지났다”라는 생각을 하게 되었습니다.

실제로 이 프로젝트는 Google Gemini를 비롯한 다양한 AI 모델과 참고 문헌을 참고하며 개발되었습니다.

이제는 단순히 코드를 작성하는 능력보다는, 독창적인 아이디어를 구상하고 이를 실현하기 위해 AI와 방대한 지식을 효과적으로 활용할 수 있는 능력이 유능한 개발자를 판단하는 중요한 기준이 될 것입니다.

프로젝트를 진행하며 C언어의 메모리 관리와 시스템 인터페이스 등 전공 지식의 필요성을 절실히 느꼈습니다.

단순히 라이브러리 함수를 사용하는 데 그치지 않고, 컴퓨터 구조론, 운영체제론 등 전공 지식이 프로그램의 근본적인 안정성과 성능에 필수적임을 깨달았습니다.

따라서 앞으로는 언어 문법뿐만 아니라, 컴퓨터공학의 핵심 지식들을 심도 있게 학습하여, 탄탄한 기초를 갖춘 실력 있는 개발자로 성장할 것을 다짐할 수 있는 계기가 되었습니다.

---

## 8. 참고문헌

인공지능 기반 개발 도구 (AI Models)

본 프로젝트는 프로그래밍 아이디어 구상과 복잡한 시스템 인터페이스 구현 과정에서 다음과 같은 인공지능 모델을 활용하여 개발 생산성을 높였습니다.

[1] Google Gemini

역할: 프로젝트 독창적 아이디어 구상, C언어 기반 핵심 게임 로직(문장 맞추기, 조기 승리 판정) 구현, 아스키 아트(ASCII Art) 캐릭터 디자인, Windows API 활용 코드 작성 및 디버깅 등 프로젝트 전반에 활용

URL: <https://gemini.google.com/>

[2] ChatGPT

역할: 게임 규칙(승패 조건) 및 보고서 초안 구조 설계, fgets와 scanf 비교 분석, 멀티미디어 기능 구현 방안 탐색 등 기술적 검증 및 자료 조사

URL: <https://chat.openai.com/>

### 8.2 외부 라이브러리 및 소스

[3] Pixabay

역할: 게임의 할로윈 테마에 맞는 배경음악(BGM) 파일을 Royalty-Free 라이선스로 확보하여 활용

URL: <https://pixabay.com/>

### 8.3 C언어 및 시스템 API 공식 문서

[4] Microsoft C/C++ 언어 참조 (MSDN)

내용: windows.h(PlaySound, Beep, Sleep, system("cls")), conio.h(\_getch) 등 Windows OS 기반 함수 사용법, #pragma comment(lib, "winmm.lib")와 같은 링커 지시문 사용법 참고

URL: <https://docs.microsoft.com/ko-kr/cpp/c-language/>

[5] C 표준 라이브러리 (<stdio.h>, <string.h> 등)

내용: strcmp(문자열 비교), fgets(입력 버퍼 처리) 등 C언어 표준 함수 동작 방식 및 매개변수 사용법 참고

URL: <https://en.cppreference.com/w/c/header>

### 개발 환경 및 사용 도구

[1] Microsoft Visual Studio 2022

역할: 통합 개발 환경(IDE)

[2] Audacity

역할: BGM 파일을 .wav 포맷으로 변환하고, 템포, 볼륨, 피치 조정 등 오디오 편집

URL: <https://www.audacityteam.org/>

---

## 9. 별첨 (소스코드와 주석)

### 1) 행맨(HangMan)

```
// C4996 (scanf/gets) 경고를 무시합니다.
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <string.h> // strcmp 함수 (단어 비교)
#include <ctype.h> // tolower 함수 등 (현재 미사용)
#include <time.h> // time 함수 (현재 미사용)
#include <windows.h> // Beep, PlaySound, Sleep, system("cls") 함수 사용
#include <conio.h> // _getch 함수 사용 (엔터 없이 입력)

// BGM 재생을 위한 윈도우 멀티미디어 라이브러리 (LNK 오류 방지)
#pragma comment(lib, "winmm.lib")

// =====
// 1. 게임 핵심 상수 정의
// =====

// 캐릭터는 총 7단계 (실패 횟수 0~6)로 구성됩니다.
#define MAX_STAGES 7
#define MAX_LINES 10

// 총 퀴즈 개수 (10문제)
#define TOTAL_QUIZZES 10
```

```
// =====  
// 2. 행맨 캐릭터 아스키 아트 데이터  
// =====
```

```
// 1. Pumpkin Style (호박)
```

```
const char* Classic_Char[MAX_STAGES][MAX_LINES] = {  
    { // Stage 0  
        " +---+ ", " | | ", " ", " | ", " ", " | ", " ", " | ", " ", " | ", " " ===""},  
    { // Stage 1: 머리 (oVo)  
        " +---+ ", " | | ", " (oVo) ", // V가 11열  
        " | ", " ", " | ", " ", " | ", " ", " | ", " " ===""},  
    { // Stage 2: 몸통 (l)  
        " +---+ ", " | | ", " (oVo) ",  
        " | | ", // 몸통 | at 11열  
        " | ", " ", " | ", " " ===""},  
    { // Stage 3: 왼팔 (/)  
        " +---+ ", " | | ", " (oVo) ",  
        " | /| ", // 왼팔 / at 10열, 몸통 | at 11열  
        " | ", " ", " | ", " " ===""},  
    { // Stage 4: 오른팔 (*)  
        " +---+ ", " | | ", " (oVo) ",  
        " | /|*", // 오른팔 * at 12열  
        " | ", " ", " | ", " " ===""},  
    { // Stage 5: 왼다리 (/)  
        " +---+ ", " | | ", " (oVo) ",  
        " | /|*",  
        " | / ", // 왼다리 / at 10열  
        " | ", " " ===""},  
    { // Stage 6: 오른다리 (i)  
        " +---+ ", " | | ", " (oVo) ",  
        " | /|*",
```

```

    "    |   /i  ", // 오른다리 i at 12열
    "    |      ", "    =====}

};

```

// 2. Skeleton Style (해골)

```

const char* Boxy_Char[MAX_STAGES][MAX_LINES] = {
    { // Stage 0
        "    +---+ ", "    | | ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    =====",
    { // Stage 1: 머리 (o)
        "    +---+ ", "    | | ", "    | (o) ", // 해골 머리
        "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    =====",
    { // Stage 2: 몸통 (H)
        "    +---+ ", "    | | ", "    | (o) ",
        "    |   H ", // 뼈 (갈비뼈)
        "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    =====",
    { // Stage 3: 왼팔 (~)
        "    +---+ ", "    | | ", "    | (o) ",
        "    |   ~H ", // 뼈 팔
        "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    =====",
    { // Stage 4: 오른팔 (~)
        "    +---+ ", "    | | ", "    | (o) ",
        "    |   ~H~ ", // 뼈 팔
        "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    =====",
    { // Stage 5: 왼다리 (l)
        "    +---+ ", "    | | ", "    | (o) ",
        "    |   ~H~ ",
        "    | | ", // 뼈 다리
        "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    |   ", "    =====",
    { // Stage 6: 오른다리 (l) - 게임 오버!
        "    +---+ ", "    | | ", "    | (o) ",
        "    |   ~H~ ",
        "    | | | ", // 뼈 다리

```

$$\left. \begin{array}{l} \text{" } | \text{ " } \text{===== " } \\ \text{" } | \text{ " } \text{===== " } \end{array} \right\}.$$

// 3. Scarecrow Style (허수아비)

```
const char* Stickman_Char[MAX_STAGES][MAX_LINES] = {
```

```
{ // Stage 0
```

"        \_ \_ \_ \_ " , "     | / | " , "     | / " , "     |        " , "     |        " , "     |        " , "     / - - - - \ \ " } ,

```
{ // Stage 1: 머리 (%)
```

" \_\_\_\_\_", " | / |", " | / @",

$$'' \quad | \quad '''' \quad | \quad '''' \quad | \quad '''' /-----\\''\},$$

```
{ // Stage 2: 몸통 (l)
```

" \_\_\_\_\_", " | / |", " | / @",

```
"      |   |", "      |   |", "      |   |", "      /-----\\", }
```

{ // Stage 3: 왼팔 (/)

" \_\_\_\_\_", " | / |", " | / @",

" | /| ", " | | ", " | ", " /-----\\\"},

{ // Stage 4: 오른팔 ( $\neg$ )

" \_\_\_\_\_", " | / |", " | / @",

```

    "    | /|¬ ", "    | | ", "    |   ", " /-----\\",

{ // Stage 5: 왼다리 (/)

    "    -----", "    | / |", "    | / @",

    "    | /|¬ ", "    | | ", "    | /   ", " /-----\\",

{ // Stage 6: 오른다리 (i)

    "    -----", "    | / |", "    | / @", // [사장님 요청] (%) -> @로 복구

    "    | /|¬ ", "    | | ", "    | /i", " /-----\\"}

};

```

```

// =====
// 3. 게임 퀴즈 데이터 (10문제)
// =====

```

```

// 문제 문장 (빈칸은 *** 로 표시)
const char* quiz_sentences[TOTAL_QUIZZES] = {
    "With great power comes *** responsibility.", // 1
    "To be or not to ***.", // 2
    "Life is *** of ups and downs.", // 3
    "Live as if you were to *** tomorrow.", // 4
    "Past is *** past.", // 5
    "Until death, it is all ***.", // 6
    "No sweat, No sw**t", // 7
    "Seeing is ***.", // 8
    "Knowledge is ***.", // 9
}

```

```
    "Time is ***."                // 10
};
```

```
// 정답 단어 (소문자)
const char* quiz_answers[TOTAL_QUIZZES] = {
    "great",    // 1
    "be",       // 2
    "full",     // 3
    "die",      // 4
    "just",     // 5
    "life",     // 6
    "ee",       // 7
    "believing", // 8
    "power",    // 9
    "gold"      // 10
};
```

```
// [추가] 문제별 힌트
const char* quiz_hints[TOTAL_QUIZZES] = {
    "힌트: 스파이더맨 명대사",
    "힌트: 햄릿",
    "힌트: '가득찬'",
    "힌트: '인간이 두려워 하는 것'",
    "힌트: '스윙스회사'",
    "힌트: '지금 살고 있는 것'",
    "힌트: 'e'",
    "힌트: '믿는 것'",
    "힌트: '힘'",
    "힌트: 노란색 덩어리"
};
```

```

// =====
// 4. 함수 원형 선언
// =====
void show_intro();
void draw_hangman(int character_choice, int stage);
void select_character();
void clear_input_buffer();
void play_game();

// =====
// 5. 함수 구현
// =====

/**
 * @brief 게임 시작 시 사용자에게 게임 규칙을 설명합니다.
 */
void show_intro() {
    printf("=====\n");
    printf("      [ 게임 설명 ]\n");
    printf("=====\n");
    printf("이 게임은 컴퓨터가 내는 %d개의 문장 빈칸을 맞추는 게임입니다.\n\n", TOTAL_QUIZZES);
    printf("패배 조건 1)\n");
    printf("단어를 6번 틀려 캐릭터가 모두 그려지면 **패배**합니다.\n\n");
    printf("패배 조건 2 / 생존 조건\n");
    printf("6번 틀리기 전에 %d문제를 모두 풀면.\n", TOTAL_QUIZZES);
    printf("맞춘 개수가 절반(6개) 이상일 경우 **생존**합니다.\n");
    printf("절반을 맞추지 못하면(5개 이하) **패배**합니다.\n");
    printf("=====\n");

    printf("\n [ 계속하려면 아무 키나 누르세요... ]");

    // 엔터 한 번으로 넘어가게 수정

```

```

int temp_char_start = getchar();
(void)temp_char_start; // 변수 사용 경고 방지

system("cls"); // 화면 지우기 (Windows 전용)
}

/**
 * @brief 선택된 캐릭터와 현재 실패 단계(stage)에 맞는 아스키 아트를 출력합니다.
 * @param character_choice 사용자가 선택한 캐릭터 (1, 2, 3)
 * @param stage 현재까지 틀린 횟수 (0 ~ 6)
 */
void draw_hangman(int character_choice, int stage) {
    if (stage < 0 || stage >= MAX_STAGES) return;

    const char** current_char_set;

    switch (character_choice) {
        case 1: current_char_set = (const char**)Classic_Char[stage]; break;
        case 2: current_char_set = (const char**)Boxy_Char[stage]; break;
        case 3: current_char_set = (const char**)Stickman_Char[stage]; break;
        default: current_char_set = (const char**)Classic_Char[stage]; break;
    }

    // 캐릭터 그림 7줄 출력
    for (int i = 0; i < 7; i++) {
        if (current_char_set[i]) {
            printf("%s\n", current_char_set[i]);
        }
    }
}

```

```

/**
 * @brief (버그 수정용) scanf, fgets 사용 후 입력 버퍼에 남은 \n(엔터)를 비웁니다.
 */
void clear_input_buffer() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

// 전역 변수로 사용자가 선택한 캐릭터 저장
int selected_char = 1;

/**
 * @brief [수정] 사용자에게 3가지 캐릭터를 보여주고 '1', '2', '3'만 입력받도록 수정
 */
void select_character() {
    char input_buffer[100];
    int choice = 0;
    int is_valid_input = 0;

    while (is_valid_input == 0) {
        system("cls"); // 화면 깨끗하게
        printf("=====\n");
        printf("    행맨(HANGMAN) 게임 캐릭터 선택\n");
        printf("=====\n\n");

        // 캐릭터 미리보기 출력 (최종 모습)
        printf(" ----- [1] 호박 (Pumpkin) -----\n");
        draw_hangman(1, MAX_STAGES - 1);
        printf("\n");
        printf(" ----- [2] 해골 (Skeleton) -----\n");
        draw_hangman(2, MAX_STAGES - 1);
        printf("\n");
    }
}

```

```

printf(" ----- [3] 허수아비 (Scarecrow) -----\n");
draw_hangman(3, MAX_STAGES - 1);
printf("\n");

printf("===== \n");
printf(" [1] 호박 (Pumpkin)\n");
printf(" [2] 해골 (Skeleton)\n");
printf(" [3] 허수아비 (Scarecrow)\n");

printf("\n 캐릭터를 선택하세요 (1, 2, 3): ");

// [수정] scanf -> fgets로 변경
if (fgets(input_buffer, sizeof(input_buffer), stdin) == NULL) {
    continue; // 입력 오류 시 다시
}

// [수정] 입력받은 문자열이 정확히 "1", "2", "3" (엔터 포함)인지 확인
if (strcmp(input_buffer, "1\n") == 0) {
    choice = 1;
    is_valid_input = 1;
}
else if (strcmp(input_buffer, "2\n") == 0) {
    choice = 2;
    is_valid_input = 1;
}
else if (strcmp(input_buffer, "3\n") == 0) {
    choice = 3;
    is_valid_input = 1;
}
else {
    // [수정] 엔터만 치거나, 다른 문자 입력 시
    printf("\n 잘못된 입력입니다. 1, 2, 3 중 하나를 정확히 입력하세요.\n");
}

```

```

        printf(" (잠시 후 다시 선택합니다...)\n");
        Sleep(1500); // 1.5초 대기
    }
} // end while

// 유효한 입력이 들어옴
selected_char = choice;
printf("\n ==> %d번 캐릭터가 선택되었습니다!\n", selected_char);

printf("\n [ 잠시 후 게임을 시작합니다... ]\n");
Sleep(1000); // 1초 대기
system("cls"); // 화면 지우기
}

```

```

/**
 * @brief 메인 게임 로직을 수행합니다. (10문제 진행)
 */
void play_game() {
    int current_quiz = 0; // 현재 문제 번호
    int wrong_guesses = 0; // 틀린 횟수 (0 ~ 6)
    int correct_answers = 0; // 맞춘 문제 수
    char user_input[100]; // 사용자 단어 입력 버퍼

    // [수정] 승리 조건 (6개)를 루프 시작 전에 정의
    int survival_threshold = TOTAL_QUIZZES / 2; // 10 / 2 = 5

    printf("\n\n#####\n");
    printf("#          [ 문장 행맨 게임 시작 ]          #\n");
    printf("#       빈칸(**)에 들어갈 단어를 맞추세요!       #\n");
    printf("#####\n");
}

```

```

// [수정] 6개를 맞추면 루프가 중단되도록 조건 추가 (correct_answers <= survival_threshold)
while (current_quiz < TOTAL_QUIZZES && wrong_guesses < MAX_STAGES - 1 && correct_answers <= survival_threshold) {

    system("cls"); // 매 턴마다 화면 깨끗하게

    printf("\n\n--- [ 문제 %d / %d ] (맞춘 개수: %d / 틀린 횟수: %d / 6) ---\n",
        current_quiz + 1, TOTAL_QUIZZES, correct_answers, wrong_guesses);

    printf("\n  문제: %s\n", quiz_sentences[current_quiz]);
    printf("\n   %s\n", quiz_hints[current_quiz]); // [추가] 힌트 출력
    draw_hangman(selected_char, wrong_guesses);

    printf("\n  빈칸(*** 또는 sw**)에 들어갈 단어를 입력하세요: ");

    if (fgets(user_input, sizeof(user_input), stdin) == NULL) {
        printf("\n입력 오류입니다.\n");
        break;
    }

    size_t len = strlen(user_input);
    if (len > 0 && user_input[len - 1] == '\n') {
        user_input[len - 1] = '\0';
    }

    if (strlen(user_input) == 0) {
        printf("\n  [X] 땡! 아무것도 입력하지 않았습니다.\n");
        printf("  [!] 정답은 '%s' 입니다.\n", quiz_answers[current_quiz]); // [수정] 정답 알려주기

        Beep(261, 300);
        Beep(261, 300);

        wrong_guesses++;
    }
}

```

```

        current_quiz++;

        printf("\n [ 정답 확인 후 다음 문제로 넘어갑니다... (1초) ]\n");
        Sleep(1000);
        continue;
    }

    if (strcmp(user_input, quiz_answers[current_quiz]) == 0) {
        // [O] 정답
        printf("\n [O] 정답입니다! '%s' 맞습니다!\n", user_input);

        Beep(523, 120);
        Beep(659, 120);
        Beep(784, 150);

        correct_answers++;
        current_quiz++;
        printf("\n [ 잠시 후 다음 문제로 넘어갑니다... (1초) ]\n");
        Sleep(1000);
    }
    else {
        // [X] 오답
        printf("\n [X] 땡! '%s'가 아닙니다...\n", user_input);
        printf(" [!] 정답은 '%s' 입니다.\n", quiz_answers[current_quiz]); // [수정] 정답 알려주기

        Beep(261, 300);
        Beep(261, 300);

        wrong_guesses++;
        current_quiz++;
        printf("\n [ 정답 확인 후 다음 문제로 넘어갑니다... (1초) ]\n");
        Sleep(1000);
    }
}

```

```

    }

} // end while loop

// =====
// 최종 결과 판정
// =====
system("cls");

printf("\n\n=====\\n");
printf("          [ 게임 결과 ]\\n");
printf("=====\\n");

// [수정] 승리/패배 판정 로직을 '조기 승리'가 가능하도록 수정

// [조건 1] 6번 틀려서 게임 오버 (최우선 패배 조건)
if (wrong_guesses == MAX_STAGES - 1) {
    printf("          게임 오버... 6번 틀렸습니다. \\n");
    printf("          *** 패 배 ***\\n");

    if (current_quiz > 0) {
        printf("      마지막 문제 정답: %s\\n", quiz_answers[current_quiz - 1]);
    }

    printf("=====\\n");
    draw_hangman(selected_char, MAX_STAGES - 1);
}

// [조건 2] 6개 이상 맞춘 경우 (생존)
else if (correct_answers > survival_threshold) { // 6~10개 맞춤
    printf("          %d개의 문제를 맞춰 생존했습니다! \\n", correct_answers);
    printf("          *** 생 존 ***\\n");
    printf("      (총 %d개 정답, %d개 오답)\\n", correct_answers, wrong_guesses);

```

```

        draw_hangman(selected_char, wrong_guesses);
    }
// [조건 3] 10문제를 다 풀었지만, 5개 이하로 맞춘 경우 (패배)
else if (current_quiz == TOTAL_QUIZZES) {
    printf("        게임 오버... 절반을 맞추지 못했습니다. \n");
    printf("                *** 패 배 ***\n");
    printf(" (총 %d개 정답, %d개 오답)\n", correct_answers, wrong_guesses);
    printf("===== \n");
    draw_hangman(selected_char, wrong_guesses);
}
}

// =====
// 6. 메인 함수 (게임 전체 루프)
// =====
int main() {

    PlaySound(L"bgm.wav", NULL, SND_ASYNC | SND_LOOP | SND_FILENAME);

    int play_again = 1;

    while (play_again == 1) {

        show_intro();

        select_character();

        play_game();

        // 재시작 여부 묻기
        printf("\n\n===== \n");
        printf("        다시 하시겠습니까?\n");
    }
}

```

```

printf("=====\n");
printf(" [1] 네 (캐릭터 선택부터 다시)\n");
printf(" [2] 아니오 (게임 종료)\n");
printf("\n 선택 (1 또는 2): ");

int choice_ch = 0;
while (choice_ch != '1' && choice_ch != '2') {
    choice_ch = _getch(); // '1' 또는 '2' 키 입력 대기
}

if (choice_ch == '1') {
    play_again = 1;
}
else {
    play_again = 0; // 루프 종료
}

system("cls");

} // end while(play_again)

// 게임 종료
printf("\n\n[ 게임을 종료합니다. 이용해주셔서 감사합니다. ]\n");

PlaySound(NULL, 0, 0);

Sleep(1500);

return 0;
}

```

